

Практическое занятие 4.

Детектирование и сопровождение объектов

Детектирование (детекция) - определение наличия на изображении и параметров объектов: координаты (бокс или описывающий прямоугольник), класс, атрибуты.

Существует фундаментальное различие между обнаружением и отслеживанием объектов

Когда мы применяем обнаружение объектов, мы определяем, где в изображении/кадре находится объект. Кроме того, детектор объектов обычно требует больших вычислительных затрат и, следовательно, работает медленнее, чем алгоритм отслеживания объектов.

Примерами алгоритмов обнаружения объектов являются каскады Хаара, HOG + линейная SVM, а также детекторы объектов на основе глубокого обучения, такие как Faster R-CNNs, YOLO и Single Shot Detectors (SSDs). Переведено с помощью DeepL.com (бесплатная версия)

С другой стороны, система отслеживания объектов принимает входные (x, y)-координаты местоположения объекта на изображении и:

- Присваивает уникальный идентификатор этому объекту
- Отслеживает объект по мере его перемещения по видеопотоку, предсказывая новое местоположение объекта в следующем кадре на основе различных атрибутов кадра (градиент, оптический поток и т. д.).

Примерами алгоритмов отслеживания объектов являются MedianFlow, MOSSE, GOTURN, ядра корреляционных фильтров, дискриминационные корреляционные фильтры и др.

Детектирование объектов в OpenCV

- Детектирование объекта по шаблону
- Детектирование лиц методом Виолы-Джонса (каскады Хаара)

Детектирование объектов с помощью нейронных сетей

- One shot детекторы
- Two shot детекторы

Трекинг - это задача отслеживания объекта в сцене.

Решается она путем предсказания местоположения объекта на последующем кадре с учетом динамики его движения.

Для сопровождения одного объекта существуют достаточно надежные методы

При отслеживании нескольких объектов необходимо отслеживать соответствие объектов текущего фрейма объектам предыдущих фреймов.

Дополнительные трудности:

- в случае их частичного перекрытия (окклюзии)

- при наличии сходства нескольких объектов между собой.

Алгоритм сопровождения объектов в видео:

1. Получить новый кадр видео.
2. Обнаружить объекты на полученном кадре.
3. Вычислить матрицу сходства между обнаруженными объектами и объектами, для которых построены отдельные части траекторий.
4. По матрице сходства ответить на следующие вопросы:
 - 4.1. Каким траекториям какой обнаруженный объект соответствует?
 - 4.2. Какие объекты появились впервые на видео (не соответствуют ни одной из уже существующих траекторий)?
 - 4.3. Для каких траекторий на новом кадре не обнаружился объект (траектория завершилась в результате выхода объекта из области видимости камерой)?
5. В соответствии с полученными ответами обновить положения в траекториях, создать новые траектории для вновь обнаруженных объектов.
6. Повторить действия, начиная с шага 1, до момента завершения видео.

Матрица сходства для траекторий и объектов – это матрица размера $N \times M$, где каждый элемент a_{ij} представляет собой коэффициент сходства траектории T_i с объектом R_j . По данной матрице можно найти наилучшее соответствие между обнаруженными объектами и существующими траекториями.

Самый простой способ вычисления коэффициента сходства траектории T_i и нового объекта R_j выглядит следующим образом:

1. Получить последнее положение объекта в траектории T_i .
2. Сравнить объект, накрываемый окаймляющим прямоугольником в последнем положении траектории, и обнаруженный объект R_j по некоторому признаку.

Для сравнения можно использовать один или несколько следующих признаков:

- расположение,
- форму,
- внешний вид объекта (размер).

Более подробно см. презентации лекций и другие материалы.

Задание 1. Детектирование объектов. Детектирование лиц с помощью фильтра Хаара

Ниже приведенная программа предназначена для детектирования лиц на изображении с помощью фильтра Хаара.

```
import cv2

# Загружаем набор данных для распознавания лица
face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
```

```

# Загружаем изображение, на котором нужно найти лицо
img = cv2.imread('test.jpg')

# Преобразуем изображение в оттенки серого
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Используем фильтры Хаара для поиска лица на изображении
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5,
minSize=(30, 30))

# Отмечаем найденные лица на изображении
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)

# Выводим результат
cv2.imshow('img', img)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

Здесь используется набор данных "haarcascade_frontalface_default.xml", содержащий информацию о признаках лица, а функция "detectMultiScale" использует этот набор данных для поиска лица на изображении. Функция возвращает координаты прямоугольной области, которая содержит найденное лицо. Затем мы используем эти координаты, чтобы нарисовать прямоугольник вокруг найденного лица на исходном изображении. Убедитесь, что у вас есть файл haarcascade_frontalface_default.xml классификатора лиц Хаара в том же каталоге, что и ваша программа. Его можно найти и скачать из Интернета.

Задание 2. Отслеживание объекта в видео ряде.

Довести ниже приведенную программу до рабочего состояния и провести эксперимент с каким-либо коротким видеороликом, например, фрагментом из какого-то фильма, каких много на Youtube.

Здесь используется простейший случай отслеживания одного наиболее крупного объекта без использования матрицы сходства.

```

import cv2
import numpy as np

# загрузка видеофайла и создание объекта видеопотока
video = cv2.VideoCapture('video.mp4')

# создание объекта фоновой вычитания
fgbg = cv2.createBackgroundSubtractorMOG2()

while True:
    # чтение кадра видеопотока
    ret, frame = video.read()

    # применение алгоритма вычитания фона
    fgmask = fgbg.apply(frame)

    # поиск контуров на двоичной карте переднего плана
    contours, hierarchy = cv2.findContours(fgmask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

```

```
# извлечение контура с максимальной площадью
max_contour = max(contours, key=cv2.contourArea)

# получение координат и размеров прямоугольника, описывающего контур
x,y,w,h = cv2.boundingRect(max_contour)

# отображение кадра со слежением за объектом
cv2.rectangle(frame, (x,y), (x+w,y+h), (0,255,0), 2)
cv2.imshow('frame', frame)

# если нажата клавиша 'q', закрыть окно
if cv2.waitKey(30) & 0xFF == ord('q'):
    break

# освобождение ресурсов
video.release()
cv2.destroyAllWindows()
```

Этот код читает файл с видеофайлом, применяет алгоритм фонового вычитания, находит контуры на двоичной карте переднего плана, извлекает контур с максимальной площадью и отображает кадр видео с прямоугольником, описывающим этот контур. Клавиша 'q' используется для выхода из программы.